

TourPlanner

Final Protocol

Tour management web application • Spring Boot + Angular • Team of 2

1. Application Overview

TourPlanner is a web application for planning and tracking tours. After registering and logging in, a user can create tours (for car, bicycle, walking, running or hiking), look at the route on a map, and write tour logs once a tour has been done. Each tour also shows two values the app computes on its own — popularity and child-friendliness — and both the tours and the logs can be searched by full text, including over those computed values. The whole catalogue can be exported to a JSON file and imported again later. On the tour detail page we also fetch the live weather at the tour's destination, which is our unique feature.

The distance and the estimated time of a tour come from the OpenRouteService API, and the map is drawn with Leaflet using OpenStreetMap tiles. Tours and tour logs always belong to the user who created them; there is no sharing between users.

Technology stack

Frontend	Angular 19 (standalone components, signals, zoneless change detection)
Backend	Spring Boot 3.4 (Web, Data JPA, Security, Validation)
Authentication	JWT (jjwt 0.12) with BCrypt password hashing
Database	PostgreSQL 16 through JPA / Hibernate
Routing	OpenRouteService directions + geocoding API
Map	Leaflet 1.9 with OpenStreetMap tiles
Weather	OpenWeatherMap API (falls back to stub data if no key)
Logging	SLF4J (Lombok <code>@Slf4j</code>) over Spring Boot's Logback backend
Configuration	Externalized in <code>application.yml</code> through environment variables
Tests	JUnit 5, Mockito and Spring MockMvc — 36 backend tests

2. Use Cases

The diagram below shows what a user can do in the application. Register and Login are the only public use cases. Everything else needs a valid JWT, so the user has to be logged in first. Because tours and logs are bound to one user, all the manage/search/import/export use cases only ever touch the data of the logged-in user.

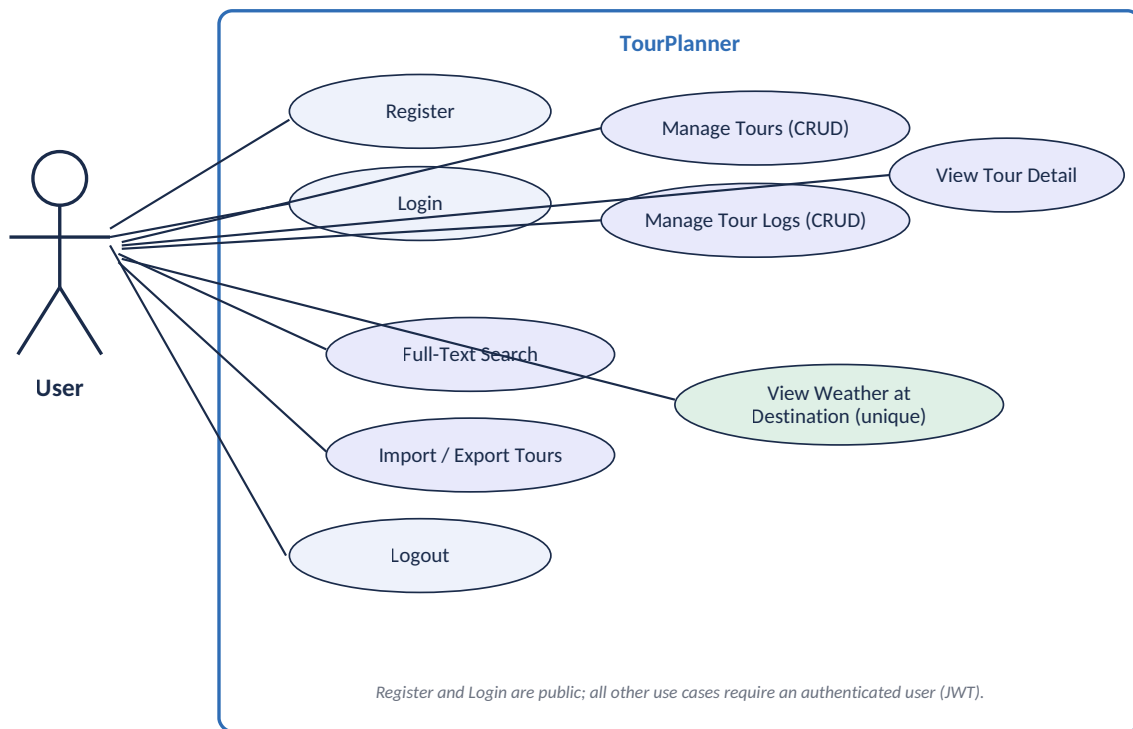


Figure 1 — Use case diagram

The CRUD use cases for tours and tour logs each validate their input on both the client and the server, so wrong input shows an error message instead of crashing anything. "View Weather at Destination" is the extra feature we added on top of the required ones.

3. Application Architecture

3.1 Layer architecture

The backend uses a classic three-layer architecture: a presentation layer (controllers), a business layer (services) and a data access layer (repositories). The rule we stuck to is that each layer only calls the layer directly below it. The controllers never touch a repository directly — they only call services. The services are the only place that talks to the repositories and to the external APIs (OpenRouteService, OpenWeatherMap). The security filter and the exception handler sit next to this flow as cross-cutting concerns.

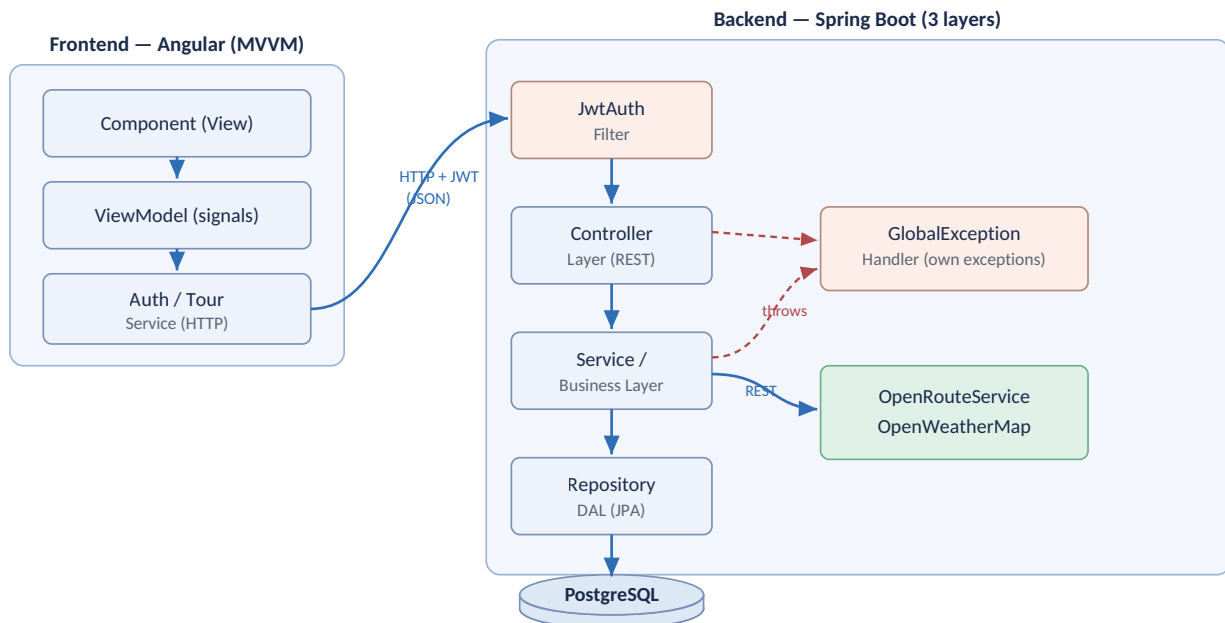


Figure 2 — Layer architecture (frontend MVVM + backend three layers)

At the controller/service boundary we map the JPA entities to DTOs with a `DtoMapper`, so the entities never leak out over HTTP. This also keeps the two computed values (popularity and child-friendliness) in one place, because the mapper is where they get calculated.

For exceptions we did not let JPA or Spring exceptions bubble up to the client. The service layer throws our own exceptions (`ResourceNotFoundException`, `BadRequestException`) and the `GlobalExceptionHandler` turns them into clean HTTP status codes — 404 for a missing or not-owned resource, 400 for bad input or failed validation. That way a database or framework exception is never shown to the user, and the layers stay decoupled from each other's implementation details.

3.2 Domain class diagram

There are three entities and two enums. A `User` owns many `Tours`, and each `Tour` has many `TourLogs`. The `Tour` uses a `TransportType` and a `TourLog` uses a `Difficulty`. The route is stored as a GeoJSON string so Leaflet can draw it directly, and the tour image is the only thing not kept in the database — it is stored on the filesystem and the entity only keeps the path to it.

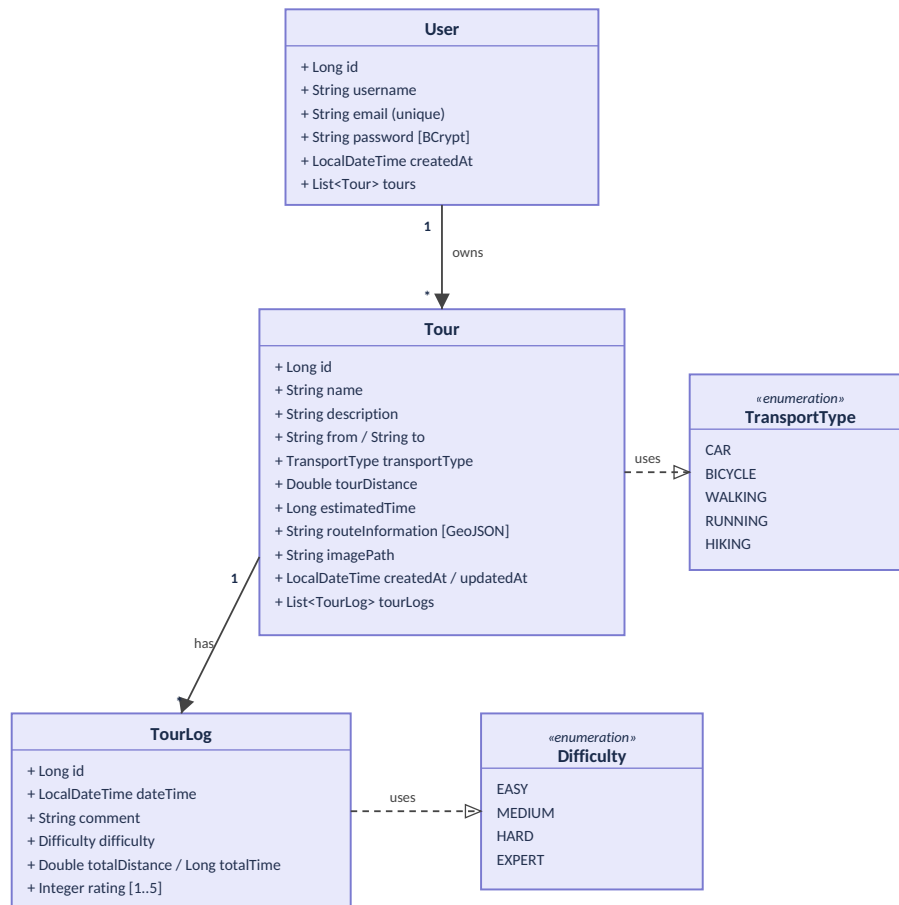


Figure 3 — Domain class diagram

3.3 Backend package structure

The packages follow the layers one-to-one, which made it easy to find things while we were working:

```

org.example.tourplanner
├── controller/          (REST API + GlobalExceptionHandler)
│   ├── AuthController    POST /api/auth/{login,register}
│   ├── TourController    /api/tours, /api/tours/{id}, /image, /export, /import
│   ├── TourLogController /api/tours/{tourId}/logs/...
│   └── GlobalExceptionHandler ResourceNotFound -> 404, BadRequest/validation -> 400
├── service/             (business logic + external integrations)
│   ├── TourService       CRUD, full-text search, import/export, image storage
│   ├── TourLogService
│   ├── AuthService       register, login, JWT issuance
│   ├── OpenRouteService  directions + geocoding (Strategy by transport type)
│   └── WeatherService     OpenWeatherMap, with stub fallback (unique feature)
├── repository/          (Spring Data JPA - DAL)
│   ├── TourRepository    findById + JPQL searchByUserAndQuery
│   ├── TourLogRepository
│   └── UserRepository
├── model/               (JPA entities: Tour, TourLog, User)
├── dto/                 (request/response shapes + DtoMapper + computed values)
└── security/            (SecurityConfig, JwtTokenProvider, JwtAuthenticationFilter,
                        UserDetailsServiceImpl)
  
```

3.4 Frontend structure (MVVM)

The frontend follows the MVVM pattern. Every feature has its own ViewModel that holds all of the state as Angular signals, and the component is just a thin view that binds to those signals. We deliberately avoided two-way `[(ngModel)]` and instead bind one way with `[ngModel]="vm.x()"` and `(ngModelChange)="vm.x.set($event)"`, so the ViewModel stays the single source of truth. The app runs with zoneless change detection, so there is no `ChangeDetectorRef` anywhere — updates are driven by the signals.

```
frontend/src/app
├── core/
│   ├── models.ts           interfaces (Tour, TourLog, User, WeatherData, ...)
│   ├── auth.utils.ts       functional authGuard + authInterceptor
│   └── services/
│       ├── auth.service.ts  register, login, token storage, isAuthenticated
│       └── tour.service.ts  tour CRUD, logs CRUD, weather, import, export
├── features/
│   ├── auth/               login.component.ts (View) + login.viewmodel.ts
│   └── tours/
│       ├── dashboard/      component + viewmodel (tours, searchQuery, onExport, onImport)
│       ├── tour-detail/    component + viewmodel (tour, logs, weather, log form state)
│       └── tour-form/       component + viewmodel (form fields, image preview, loading)
└── shared/
    ├── navbar.component.ts  (reused on dashboard, detail and form)
    ├── tour-map.component.ts (Leaflet wrapper - our reusable component)
    ├── auth-image.pipe.ts   (adds the Bearer token to image URLs)
    └── format.utils.ts      (formatDistance, formatTime, getTransportLabel)
```

4. UI Flow & Wireframes

The application has four screens. Login is the entry point, the dashboard is the central hub, and the tour detail and tour form are both reached from the dashboard. The navbar at the top of every authenticated screen has the logout button.

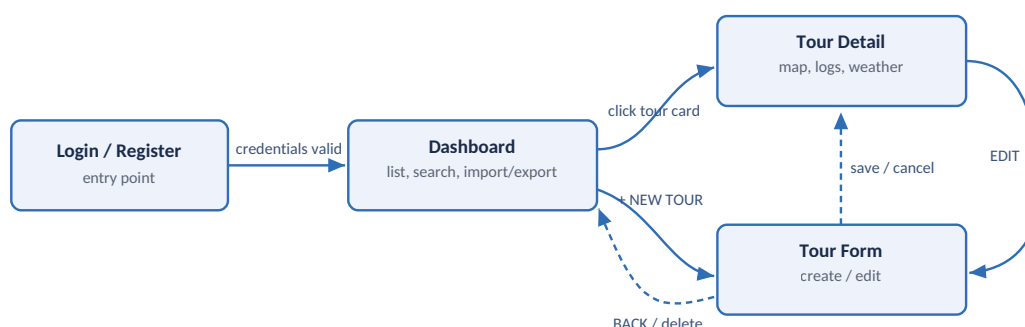
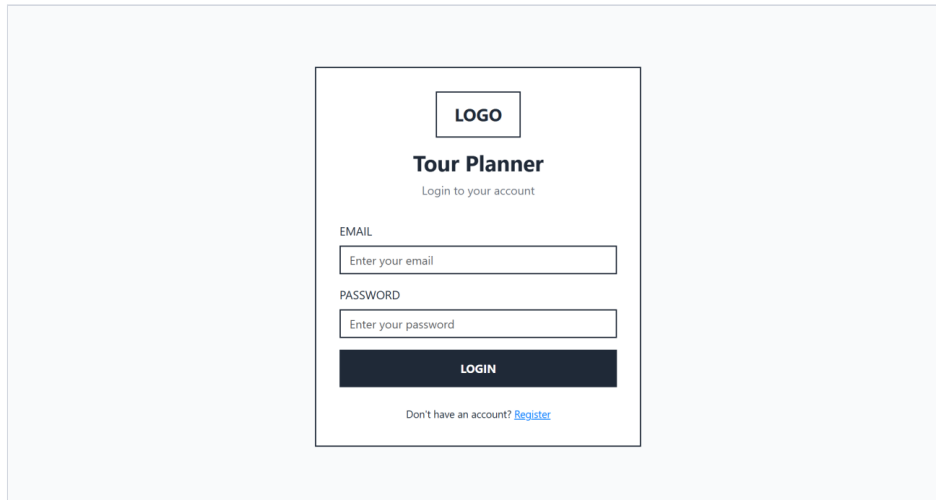


Figure 4 — UI flow between the four screens

The wireframes below are the ones from our intermediate protocol. The final UI follows them, with a few additions we made for the final hand-in: the **Weather at Destination** card on the tour detail page, the wired-up **Import** and **Export** buttons on the dashboard (with an inline error banner when a file fails to parse), and a search placeholder that names what is actually searched (name, description, route, log comment, popularity).

4.1 Login / Register

A centered card with the logo, the form fields and a toggle to switch between login and registration. In registration mode it also asks for a name and a confirm-password field.

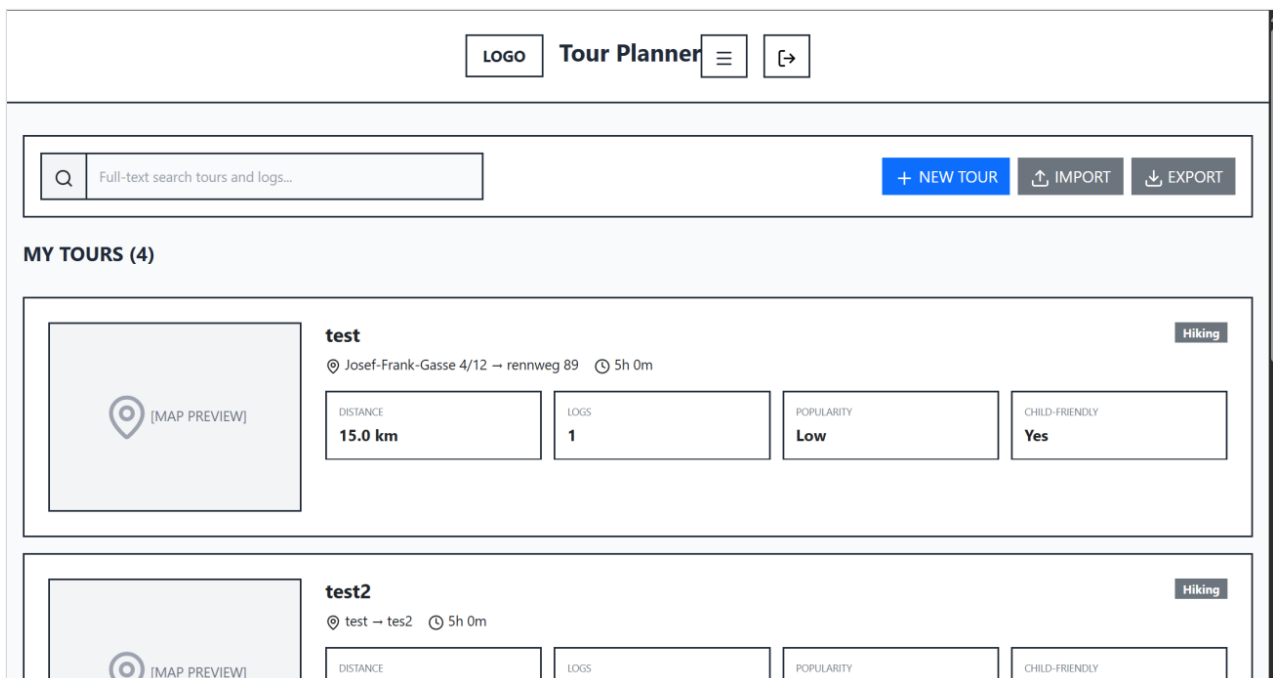


The mockup shows a centered card on a light gray background. At the top of the card is a box labeled 'LOGO'. Below it is the title 'Tour Planner' in bold, followed by the subtitle 'Login to your account'. There are two form fields: 'EMAIL' with the placeholder 'Enter your email' and 'PASSWORD' with the placeholder 'Enter your password'. Below these fields is a dark blue button labeled 'LOGIN'. At the bottom of the card, there is a link: 'Don't have an account? [Register](#)'.

Figure 5 — Login / Register screen

4.2 Dashboard

The dashboard shows the navbar, an action bar with the full-text search box and the New Tour / Import / Export buttons, and a list of tour cards. Each card has a map preview, the tour name, the route, the transport badge and a small stats grid (distance, number of logs, popularity, child-friendly).



The dashboard mockup features a top navigation bar with a 'LOGO' box, the text 'Tour Planner', a hamburger menu icon, and a share icon. Below this is an action bar containing a search box with the placeholder 'Full-text search tours and logs...' and three buttons: '+ NEW TOUR' (blue), 'IMPORT' (gray), and 'EXPORT' (gray). The main content area is titled 'MY TOURS (4)'. It displays two tour cards. The first card is for 'test' with a route 'Josef-Frank-Gasse 4/12 → rennweg 89' and a duration of '5h 0m'. It has a 'Hiking' badge and a stats grid showing 'DISTANCE: 15.0 km', 'LOGS: 1', 'POPULARITY: Low', and 'CHILD-FRIENDLY: Yes'. The second card is for 'test2' with a route 'test → tes2' and a duration of '5h 0m', also with a 'Hiking' badge and a similar stats grid. Each card includes a map preview icon.

Figure 6 — Dashboard screen

4.3 Tour Detail

The tour detail page shows all the attributes of the tour, the route on a Leaflet map, the computed stats, and the full list of tour logs which can be added, edited and deleted inline. In the final version the Weather at Destination card sits under the stats grid.

← BACK

EDITDELETE

test

Hiking

test

📍 Josef-Frank-Gasse 4/12 → rennweg 89 ⌚ Estimated: 5h 0m

DISTANCE

15.0 km

EST. TIME

5h 0m

POPULARITY

Low

(from 1 logs)

CHILD-FRIENDLY

Yes

(computed)



TOUR LOGS (1)

+ ADD LOG

📅 03/09/2026 at 18:11 #1

daf1

Difficulty: MEDIUM

Distance: 11.0 km

Time: -

Rating: ★

EDITDELETE

Figure 7 — Tour detail screen (the final version adds a weather card under the stats)

4.4 Tour Create / Edit Form

One form component is reused for both creating and editing a tour. It is split into Basic Information, Route Information (with the transport type selector), Distance & Time, and a live map preview. Required fields are marked with a star and validated before the form can be submitted.

← BACK

CREATE NEW TOUR

BASIC INFORMATION

TOUR NAME *

Enter descriptive tour name

Required field

DESCRIPTION *

Describe your tour in detail

Required field

ROUTE INFORMATION

FROM (START LOCATION) *

TO (END LOCATION) *

Enter start location

Enter end location

TRANSPORT TYPE *

Bike

Hiking

Running

Vacation

DISTANCE & TIME

TOUR DISTANCE (km)

ESTIMATED TIME (seconds)

Enter distance in km

Enter time in seconds

ROUTE MAP PREVIEW

VALIDATION

- All fields marked with * are required
- Start and end locations must be valid addresses
- Tour name must be unique
- Form validates input before submission

CREATE TOUR

CANCEL

Figure 8 — Tour create / edit form

4.5 Reusable component: TourMap

The required reusable UI component is `TourMapComponent`, a wrapper around Leaflet. It takes `routeGeoJson`, `fromLocation` and `toLocation` as inputs and handles the map setup, drawing the GeoJSON route, and fitting the bounds. We use the exact same component in two places — the live preview

in the tour form and the full route on the tour detail page. The navbar is a second shared component used on every authenticated page.

5. Full-Text Search (Sequence Diagram)

Full-text search was the trickiest feature because the search has to cover the computed values too, and those can't be expressed in SQL since they are derived from the logs. We solved it by splitting the work: a JPQL query handles the stored text fields, and then the service does a second in-memory pass for popularity and child-friendliness. The frontend debounces the input by 300 ms so we don't fire a request on every keystroke.

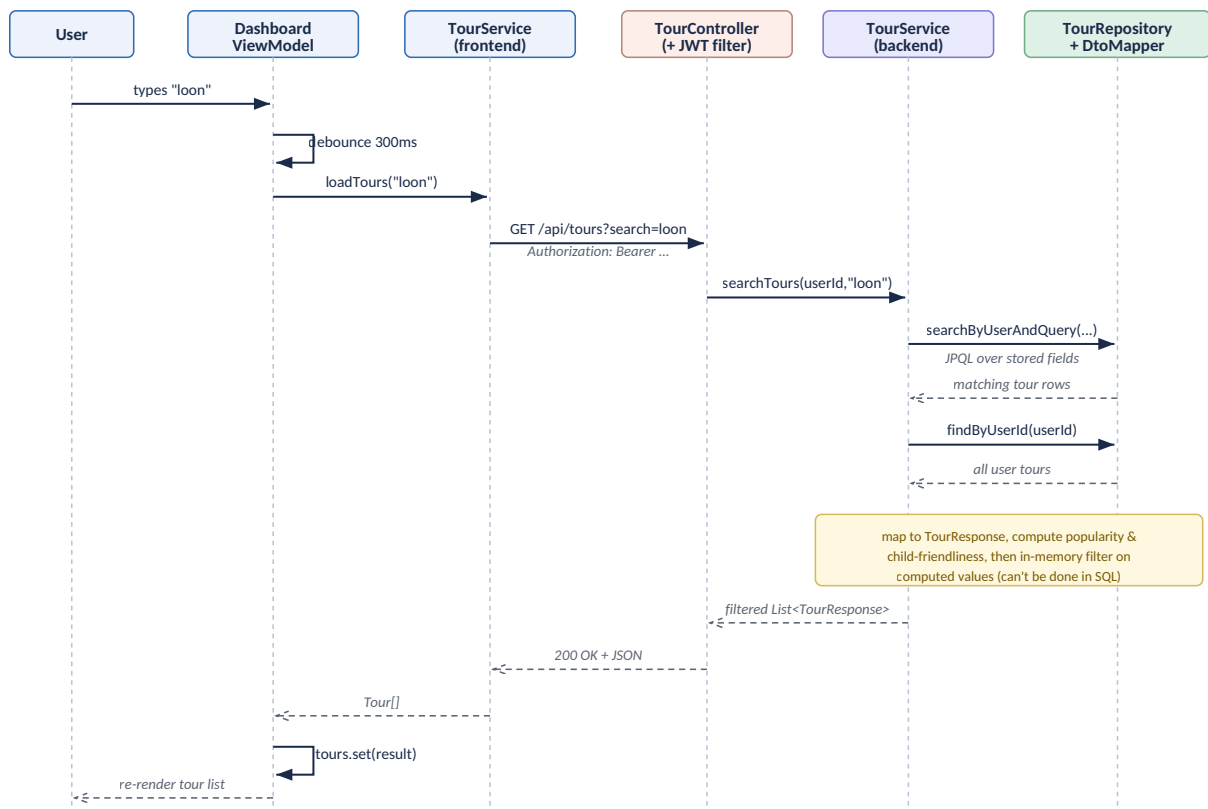


Figure 9 — Sequence diagram for full-text search

Implementation references: `dashboard.viewmodel.ts` (debounce), `tour.service.ts` → `loadTours(search?)`, `TourRepository.java` (JPQL `searchByUserAndQuery`), `TourService.searchTours` (the in-memory pass), and `DtoMapper` (popularity and child-friendliness formulas).

6. Design Patterns

We used several patterns, most of them because Spring or Angular pushed us towards them and they genuinely made the code cleaner:

Pattern	Where we used it	Why
Repository	TourRepository, TourLogRepository, UserRepository extend <code>JpaRepository<Entity, Long></code>	Hides the JDBC and gives us derived queries (<code>findByUserId</code>) and a JPQL query for free,

		with parameter binding so there is no SQL injection.
DTO + Mapper	dto/*Request, dto/*Response, DtoMapper	Keeps entities off the HTTP surface and puts the computed-value logic in one place.
Strategy	<code>OpenRouteService.PROFILE_MAP</code> maps <code>TransportType</code> to an ORS profile (HIKING → "foot-hiking", etc.)	Adding a transport type is one map entry instead of another <code>switch</code> branch.
Builder (Lombok)	<code>@Builder</code> on <code>Tour</code> , <code>TourLog</code> , <code>User</code> and the response DTOs	Makes the test fixtures readable, e.g. <code>Tour.builder().name(...).build()</code> .
Dependency Injection	<code>@RequiredArgsConstructor</code> + Spring DI everywhere	Collaborators come in through the constructor as final fields, which makes them trivial to mock in the tests.
MVVM (frontend)	component (View) + viewmodel (signals) per feature	State lives in the ViewModel, the component only renders. Required by the assignment.

7. Unique Feature — Weather at Destination

Our unique feature is live weather at the tour's destination, using the OpenWeatherMap API. It is built end to end:

- **Backend:** `WeatherService` calls the OpenWeatherMap current-weather endpoint with the geocoded destination, parses the answer into a `WeatherResponse` DTO, and exposes it at `GET /api/tours/{tourId}/weather`. If no `WEATHER_API_KEY` is configured it returns plausible stub data, so the demo still works offline.
- **Frontend:** the tour detail `ViewModel` loads the weather when the page opens and stores it in a `weather` signal. The "Weather at Destination" card shows the temperature, the condition, the OpenWeatherMap icon, humidity and wind.
- **Resilience:** the weather call is non-blocking. If it fails, the error handler just clears the signal and the rest of the page keeps working without a weather card.

We picked this because it goes past the required tour-management features and pulls in a third external service besides `OpenRouteService`, and the fallback behaviour shows the app degrading nicely instead of breaking.

8. Logging & Configuration

Logging. The backend logs through SLF4J on top of Logback. We added Lombok's `@Slf4j` to the three classes that do external I/O — `OpenRouteService`, `WeatherService` and `TourService` — so every way an integration can fail leaves a trace without cluttering the happy path. We log at the boundary where an external system can fail and pick the level by how bad it is: `error` when a required integration is unusable (missing ORS key, a failed directions call), `warn` when the app still works in a degraded way (missing weather key then stub fallback, geocoding failure during a weather lookup), and `info` for normal business events like a duplicate skipped on import. The levels are set in `application.yml` so we can tune the verbosity without touching the code.

Configuration. No connection string, secret or base directory is hard-coded. Every environment-specific value in `application.yml` is written as `${ENV_VAR:default}` — the datasource (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`), the `JWT_SECRET`, the two API keys (`ORS_API_KEY`, `WEATHER_API_KEY`) and the `UPLOAD_DIR` where the tour images are stored. The defaults that are committed are only for local development; a real deployment overrides them from the environment so no production credentials end up in the source.

9. Library Decisions & Lessons Learned

- **JWT instead of server sessions.** Stateless tokens fit the SPA-plus-REST setup, keep the backend horizontally scalable and avoided cookie/CORS headaches while developing locally. The downside is that revoking a token needs a denylist, which we decided was fine for a personal tour app.
- **Leaflet instead of Google Maps.** It's open source, works with OpenStreetMap, needs no API key, and it eats the GeoJSON from OpenRouteService directly with `L.geoJSON()`, so there was nothing to convert.
- **OpenRouteService quirk.** ORS wants lon/lat coordinates, not place names. So every tour save first geocodes both ends and then asks for directions — two calls per save. We wrapped both calls in one method so the callers don't have to care.
- **The computed-value search.** This was the lesson that took us the longest. We first tried to do the whole search in SQL and got stuck, because popularity and child-friendliness don't exist as columns. Splitting it into a JPQL query plus an in-memory filter was the fix and it keeps the search fast while still being correct.
- **Import being idempotent.** We chose to silently skip tours with a duplicate name on import instead of erroring out. Re-importing the same file is something people do, and stopping the whole import for one duplicate row, or overwriting the user's edits, both felt worse than just skipping.
- **Lombok.** `@Data`, `@Builder` and `@RequiredArgsConstructor` cut a lot of boilerplate. The cost is the extra annotation processor, which we excluded from the runnable JAR in the Maven plugin config.

10. Unit Tests

We wrote 36 backend tests across 9 test files. The goal was to pin down one specific behaviour per test and to cover every layer that actually contains logic. We preferred plain unit tests with `@ExtendWith(MockitoExtension.class)`, and the controller tests use `@WebMvcTest` with mocked services so they boot in milliseconds. No test shares a fixture with another — each one sets up its own state in `@BeforeEach`.

Test file	#	What it checks
AuthControllerTest	2	register and login return 200 with the right response shape
TourControllerTest	3	GET /tours, GET /tours/{id}, POST /tours produce the expected JSON
AuthServiceTest	4	register hashes the password, login validates it, duplicate user rejected, unknown user gives 401
TourServiceTest	8	list, get, create, update, delete; the ownership check; mapping the ORS result into the entity
TourServiceSearchTest	4	

		search hits name, description and the computed popularity; a blank query returns everything without calling JPQL
TourServiceImportExportTest	3	export round-trips the logs; import creates tour + logs; import skips duplicate names
TourLogServiceTest	5	log CRUD; a log stays scoped to its tour; the ownership check
DtoMapperTest	4	tour-to-response mapping; popularity thresholds (0/1/5/10); child-friendliness from logs
JwtTokenProviderTest	4	token signs, verifies, extracts the subject, and rejects tampered or expired tokens
Total	36	

Why these and not others: the service-layer tests are the highest value because they hold the business rules everything else trusts — a bug here could let a tour cross a user boundary or accept an invalid log.

`DtoMapperTest` matters because the popularity and child-friendliness formulas are the one place the spec asks for computed values, and if a formula silently breaks then both the search and the UI start lying.

`JwtTokenProviderTest` guards the security boundary that every "pretend the user is logged in" test depends on. `TourServiceSearchTest` locks in the JPQL-plus-in-memory search so the rubric line about searching the computed values keeps holding, and `TourServiceImportExportTest` stops two specific bugs: silently losing logs on a round-trip, and silently overwriting a tour on a duplicate import.

11. Tracked Time

These hours are estimated from our git history (33 commits between 16 Feb and 18 May 2026). They are a best-effort reconstruction rather than a stopwatch record.

Phase	Dates	Hours
Project setup; backend domain model + repositories	Feb 16 – Feb 21	10
Auth + JWT + DTOs	Feb 23 – Feb 26	8
Services: TourService, OpenRouteService, WeatherService	Feb 25 – Feb 28	8
REST controllers + GlobalExceptionHandler	Feb 28 – Mar 1	6
Backend unit tests (first set)	Mar 1	5
Frontend init: project, models, core services	Mar 2 – Mar 3	6
Auth UI + navbar + reusable TourMap component	Mar 4 – Mar 5	6
Dashboard, tour detail, tour form (create/edit)	Mar 6 – Mar 9	12
Intermediate feedback (search, map, nav fixes)	Apr 3 – Apr 9	6
Final features: search expansion, import/export, weather UI, navbar consistency	Apr – May	10
Logging, config externalization, bug fixes	May 17 – May 18	4
Final protocol: UML diagrams, pattern write-up	May	6

12. Git Repository

The repository is at https://github.com/Many-Attempts/tour_planner.git

The git history is part of the documentation. Every commit message describes the incremental step it adds (for example *add Tour and TourLog domain models*, *add OpenRouteService and weather API integration*, *add backend unit tests*). The state of the intermediate hand-in is the commit right before the final work begins (`10ada24 fixed nav`).